HospiTrack Deployment Configuration Summary

This document provides an overview of all deployment-related files created for production-ready deployment.

Files Created

1. Docker Configuration

`Dockerfile.prod`

- **Purpose**: Production-optimized Docker image
- **Features**:
- Multi-stage build for smaller image size
- Non-root user for security
- Health check instruction
- Optimized layer caching
- 4 Gunicorn workers with Uvicorn
- **Base image**: `python:3.11-slim`
- **Final user**: `hospitrack` (non-root)

`.dockerignore`

- **Purpose**: Exclude unnecessary files from Docker builds
- **Excludes**: Git files, Python cache, tests, docs, IDE configs, logs

`docker-compose.prod.yml`

- **Purpose**: Production deployment with Docker Compose
- **Features**:
- Resource limits (2GB RAM, 2 CPU cores)
- Health checks
- Restart policies (always)
- Log rotation
- Network isolation

2. Platform Configuration

`render.yaml`

- **Purpose**: Render.com Blueprint configuration
- **Features**:
- Web service definition
- Auto-deploy from GitHub
- Environment variable templates
- Health check path
- Scaling configuration (1-3 instances)
- **Runtime**: Docker
- **Plan**: Starter (configurable)

3. Environment Variables

`.env.example`

- **Purpose**: Environment variable template
- **Categories**:
- Application settings (PORT, PYTHONUNBUFFERED)
- Data configuration (HOSPITRACK_DATA_PATH)
- Geocoding settings (CACHE_SIZE, RATE_LIMIT)
- Feature flags (ML_DEMO_ENABLED)
- Logging (LOG_LEVEL)
- Performance (GUNICORN_WORKERS, TIMEOUT)
- Security (ALLOWED_HOSTS, CORS_ORIGINS)
- Monitoring (SENTRY_DSN)

4. Documentation

`DEPLOYMENT.md` (17.5 KB)

- **Purpose**: Comprehensive deployment guide
- **Sections**:
- Render deployment (primary)
- Fly.io deployment
- Railway deployment
- Docker Compose on VPS
- Custom VPS/Cloud (AWS, GCP, Azure)
- Nginx reverse proxy setup
- SSL/TLS configuration
- Post-deployment verification
- Troubleshooting

`DEMO_SCRIPT.md` (14.5 KB)

- **Purpose**: Product demo walkthrough
- **Sections**:
- Pre-demo checklist
- Landing page walkthrough
- Emergency care search demo
- Results page deep dive
- Explore page demo
- Company demo page
- Key talking points
- Q&A handling

`PRODUCTION_CHECKLIST.md` (12.6 KB)

- **Purpose**: Pre-deployment verification checklist
- **Categories**:
- Pre-deployment checks (code quality, data, config, testing)
- Security checks (app security, Docker security, data privacy)
- Deployment execution

- Post-deployment verification
- Operational readiness
- Compliance & legal
- Performance optimization
- Monitoring & maintenance

`README.md` (Updated, 25.9 KB)

- **Purpose**: Complete project documentation
- **Sections**:
- Overview and key features
- Quick start guide
- Detailed feature descriptions
- Installation instructions
- Usage examples (web + CLI)
- API documentation
- Data management
- Deployment overview
- Testing instructions
- Environment variables
- Contributing guidelines
- Architecture overview
- Troubleshooting
- License and credits

5. Linting & Formatting Configuration

`.flake8`

- **Purpose**: Python linting rules
- **Settings**:
- Max line length: 100
- Max complexity: 15
- Ignores: E203, E501, W503 (Black-compatible)

`pyproject.toml`

- **Purpose**: Python project configuration
- **Tools configured**:
- Black (code formatter)
- isort (import sorter)
- pytest (test runner)
- coverage (test coverage)

`.prettierrc`

- **Purpose**: JavaScript/CSS formatting rules
- **Settings**:
- Print width: 100
- Tab width: 2 spaces
- Single quotes: false
- Trailing commas: ES5

`.pre-commit-config.yaml`

- **Purpose**: Pre-commit hooks configuration
- **Hooks**:
- Black (Python formatting)
- isort (import sorting)
- Flake8 (Python linting)
- Prettier (JS/CSS formatting)
- Hadolint (Dockerfile linting)
- General file checks (trailing whitespace, EOF, YAML/JSON validation)

6. CI/CD Pipeline

`.github/workflows/ci.yml`

- **Purpose**: GitHub Actions CI/CD workflow
- **Jobs**:
  1. **Lint**: Black, isort, Flake8, Prettier
  2. **Test**: pytest with coverage reporting
  3. **Build**: Docker image build with caching
  4. **Security**: Safety check, pip-audit
  5. **Deploy**: Trigger Render deploy (on main branch)
  6. **Health Check**: Post-deployment verification
- **Triggers**: Push to main/develop, pull requests to main

Files Summary

Created Files (11)

1. `Dockerfile.prod` - Production Docker image
2. `.dockerignore` - Build exclusions
3. `docker-compose.prod.yml` - Production compose
4. `render.yaml` - Render platform config
5. `.env.example` - Environment variable template
6. `.flake8` - Python linting config
7. `pyproject.toml` - Python project config
8. `.prettierrc` - JS/CSS formatting config
9. `.pre-commit-config.yaml` - Pre-commit hooks
10. `.github/workflows/ci.yml` - CI/CD pipeline
11. `DEPLOYMENT_SUMMARY.md` - This file

Updated Files (1)

1. `README.md` - Comprehensive documentation (fully rewritten)

New Documentation (3)

1. `DEPLOYMENT.md` - Deployment guide (17.5 KB)
2. `DEMO_SCRIPT.md` - Product demo script (14.5 KB)
3. `PRODUCTION_CHECKLIST.md` - Pre-deployment checklist (12.6 KB)

Total New Content

- **15 files** created/updated

- **~65 KB** of documentation
- **200+ lines** of configuration code
- **6 CI/CD jobs** configured
- **4 deployment platforms** documented

Security Features

- ✅ Non-root user in Docker container
- ✅ Minimal base image (python:3.11-slim)
- ✅ Multi-stage build (reduces attack surface)
- ✅ .dockerignore excludes sensitive files
- ✅ Environment variables externalized
- ✅ Health checks enabled
- ✅ Security scanning in CI/CD
- ✅ No hardcoded secrets
- ✅ HTTPS enforced in production docs
- ✅ Medical disclaimers present

Performance Optimizations

- ✅ Parquet file for fast data loading
- ✅ Gunicorn with multiple workers
- ✅ LRU geocoding cache
- ✅ Docker layer caching in builds
- ✅ Resource limits prevent runaway usage
- ✅ Log rotation configured
- ✅ Health check intervals optimized

Next Steps for Deployment

1. Update Placeholders

- Replace `YOUR_USERNAME` with actual GitHub username in README.md
- Update `https://your-app.onrender.com` with actual deployment URL

2. Set Up GitHub Repository

```
git add .
git commit -m "Add production deployment configuration"
git push origin main
```

3. Deploy to Render

1. Go to render.com (https://render.com) and connect GitHub
2. Click “New +” → “Blueprint”
3. Select `hospitracker` repository
4. Render will detect `render.yaml` and auto-configure
5. Set environment variables in dashboard

6. Click “Apply” to deploy

4. Verify Deployment

```
# Health check
curl https://your-app.onrender.com/healthz

# Test API
curl https://your-app.onrender.com/docs
```

Status: ✅ All deployment configuration files created and validated

Last Updated: December 15, 2024